

# Bus Reservation And Ticketing System

<https://github.com/aiyshawry/Bus-Reservation-and-Ticketing-System>

Java    OOP    Console Application

## OVERVIEW

---

A Java-based console application that simulates a complete bus ticket reservation and billing workflow, supporting multi-passenger booking, seat management, and discount-based billing operations.

## IMPACT

---

Built a fully functional console-based Bus Reservation and Ticketing System in Java with structured input validation, in-memory seat management, and automated billing with passenger-type discounts.

## TECHNICAL WALKTHROUGH

---

“I built a console-based Bus Reservation and Ticketing System in Java that allows seat management, passenger booking, billing with discounts, and transaction tracking using structured input validation and in-memory data storage. ”

### Problem statement

Manual ticket booking is error-prone and slow.

This project simulates a real-world bus reservation system where users can manage destinations, seat availability, passenger bookings, billing, and payment status through a menu-driven interface.

### High-level system flow

Explain this in steps:

#### Step 1: Authentication

Simple login using username and password

Prevents unauthorized access

“This simulates role-based access at a basic level.”

#### Step 2: Main Menu

The system has 5 main modules:

Destination Management

Passenger Booking

Billing & Payment

View Passenger Details

Exit

### Module-wise explanation

#### 1. Destination Module

Displays available destinations

Shows:

Fare per destination

Remaining seat count

**Seats are initialized dynamically**

“Seat availability updates in real time after each booking.”

## **2. Passenger Booking Module**

This is the core logic.

**What happens here:**

Passenger name validation (no duplicates)

**Destination selection with validation**

**Seat availability check**

**Passenger count input**

**Discount handling**

**Fare calculation**

**Business logic:**

Total Fare = (Regular passengers × fare) + (Discounted passengers × (fare – 20%)) “This ensures accurate billing while enforcing seat limits.”

## **3. Billing & Payment Module**

**Search passenger by name**

**Prevent double payment**

**Accept payment**

**Compute change**

**Mark passenger as PAID**

“Once paid, the passenger status is locked to avoid duplicate billing.”

## **4. View Module**

**Search passenger details**

**Displays:**

**Destination**

**Passenger count**

**Discount count**

**Total fare**

**Payment status**

Amount paid and change (if applicable) “This acts like a transaction history lookup.”

## **5. Exit Module**

**Gracefully exits the system**

**Confirms continuation**

## **KEY DETAILS**

---

- Implements user authentication, seat selection, and multi-passenger booking in a single session.

- Handles billing logic including discount tiers for different passenger categories.

- Uses structured in-memory data storage for transaction tracking without a database dependency.

- Validates all inputs to prevent double-booking and illegal seat selections.